

Implementing Threads in the Kernel

Definition

Kernel-level thread implementation means the kernel knows about threads and manages them directly. Thread creation, destruction, blocking, and scheduling are handled by the kernel rather than by a user-space run-time library.

1. Basic Idea

Kernel-Managed Threads

In a kernel-thread system, there is no separate run-time thread system inside each process for core thread management. The kernel itself maintains the thread data structures and performs thread operations.

No Per-Process Thread Table

Unlike user-level threads, there is no private thread table inside every process. Instead, the kernel keeps a **system-wide thread table**.

2. Kernel Thread Table

Central Thread Table

The kernel thread table stores information about every thread in the system.

Typical Stored Information

The kernel thread table contains per-thread data such as:

- registers,
- thread state,
- program counter,
- and other execution-related information.

Relation to the Process Table

The kernel still keeps the ordinary process table as well. The process table tracks processes, while the kernel thread table tracks individual threads.

3. How Thread Operations Work

Kernel Calls

When a thread wants to create or destroy another thread, it must make a **kernel call**. The kernel then updates the kernel thread table accordingly.

Cost Implication

Because thread operations require entering the kernel, they are more expensive than user-level thread operations that stay entirely in user space.

4. Blocking Behavior

Major Advantage

If a thread blocks in the kernel, the kernel can choose another runnable thread to execute.

What the Kernel Can Do

When one thread blocks, the kernel may schedule:

- another thread from the **same process**, or
- a thread from a **different process**.

Contrast with User-Level Threads

With user-level threads, the run-time system can only keep switching among threads inside its own process until the kernel takes the CPU away. With kernel threads, the kernel directly decides what runs next.

5. Advantages of Kernel Threads

Advantage	Explanation
Blocking system calls work naturally	If one thread blocks, others may still run
Page faults are handled better	A page fault in one thread does not necessarily stop the whole process
True kernel scheduling	The kernel can pick the best runnable thread system-wide
No wrapper tricks needed	No need for jackets or wrappers around blocking calls

No Need for New Nonblocking Calls

Kernel threads do not require the operating system to invent special nonblocking versions of common system calls just to keep other threads running.

6. Page Fault Advantage

Better Than User-Level Threads

If one thread causes a page fault, the kernel can check whether another thread in the same process is runnable and execute it while waiting for the missing page to be loaded from disk.

Why This Matters

This avoids one of the major weaknesses of user-level threads, where a page fault caused by one thread blocks the entire process.

7. Thread Recycling

Recycling Idea

Creating and destroying kernel threads is relatively expensive. To reduce this overhead, some systems **recycle** threads.

How Recycling Works

When a thread is “destroyed,” the system may simply mark it as not runnable and keep its kernel data structures. Later, instead of building a new thread from scratch, the kernel reactivates that old thread structure.

Why More Useful Here

Thread recycling is possible with user-level threads too, but it is much more useful for kernel threads because kernel thread creation and destruction cost more.

8. Main Disadvantage

Higher Overhead

The main drawback of kernel threads is that thread operations such as creation, destruction, and blocking require system calls, and system calls are relatively expensive.

Practical Result

If thread operations happen frequently, the extra kernel overhead can become significant.

9. Problems Kernel Threads Do Not Solve

9.1 Problem 1: `fork`

Fork Ambiguity

If a multithreaded process calls `fork`, it is not always obvious what the child process should contain.

Two Possible Answers

- The child could receive only one thread.
- The child could receive copies of all the parent’s threads.

Why the Best Choice Varies

If the child will immediately call `exec`, then copying only one thread may be best. If the child continues running the same program, copying all threads may be more appropriate.

9.2 Problem 2: Signals

Signal Delivery

In the classical model, signals are sent to processes, not to individual threads. That raises a question: when a signal arrives, which thread should handle it?

Possible Approach

Threads might register interest in particular signals, so that a signal is sent to the thread that requested it.

New Ambiguity

What happens if two or more threads register for the same signal? The design becomes complicated again.

10. Kernel Threads vs User-Level Threads

Feature	User-Level Threads	Kernel Threads
Who manages them?	User-space library	Kernel
Thread table location	Inside each process	In the kernel
Switch speed	Very fast	Slower, due to kernel involvement
Blocking system calls	Can block whole process	Usually block only the calling thread
Page fault effect	Can block whole process	Kernel can run another thread
Overhead	Low	Higher

11. Hybrid Implementations

Main Idea

A **hybrid implementation** tries to combine the advantages of user-level threads and kernel-level threads.

How It Works

One common hybrid design uses **kernel-level threads** as the units known to the kernel, and then **multiplexes user-level threads** onto some or all of those kernel threads.

Flexibility

This model gives the programmer control over:

- how many kernel threads to create,
- how many user-level threads to place on each kernel thread.

Because of this, it offers a very flexible design.

Kernel View

In a hybrid system, the kernel is aware only of the **kernel-level threads**. It schedules those threads, not the user-level threads placed on top of them.

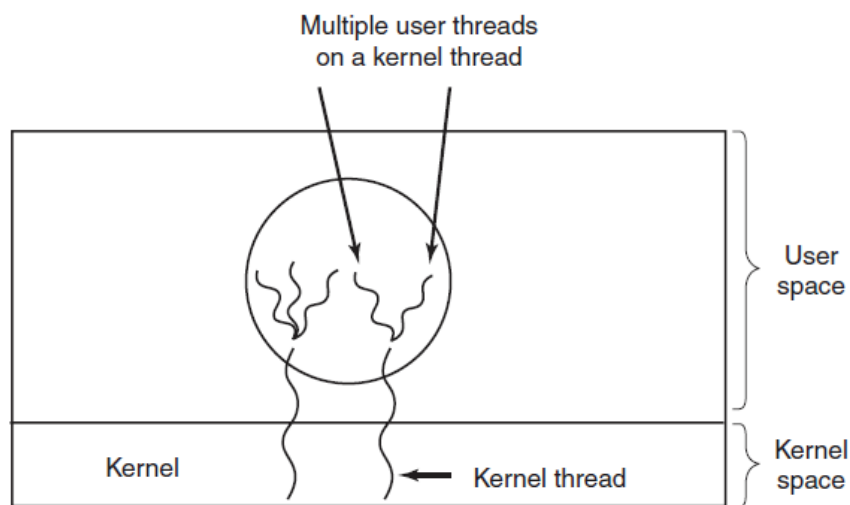


Figure 1: Multiplexing user-level threads onto kernel-level threads.

User-Level Scheduling on Top

The user-level threads running on top of a kernel thread are created, destroyed, and scheduled much like user-level threads in a process running on an operating system with no direct multithreading support.

Key Picture

You can think of the model as:

- the kernel scheduling a smaller number of kernel threads,
- while each kernel thread acts like a carrier for one or more user threads that take turns using it.

Feature	User-Level Threads	Kernel Threads	Hybrid Model
Who the kernel sees	Process only	Every thread	Kernel threads only
Who manages user execution	User library	Kernel	Both kernel and user library
Blocking behavior	Weak for blocking calls	Stronger	Depends on mapping and design
Flexibility	High inside one process	Lower-level OS control	Highest, but more complex

Main Tradeoff

Hybrid implementations provide great flexibility, but the design becomes more complex because thread management is split between user space and the kernel.

12. Main Conclusion

Takeaway

Kernel threads solve the biggest practical problems of user-level threads, especially blocking system calls and page faults. Their cost is higher overhead, and they still leave difficult design questions such as **fork** behavior and signal handling unresolved. Hybrid implementations try to combine the strengths of both approaches by multiplexing user-level threads onto kernel-level threads.

13. Quick Review

Exam-Focused Points

- In kernel threads, the kernel knows about and manages all threads.
- The kernel keeps a system-wide thread table in addition to the process table.
- Thread creation and destruction are done through kernel calls.
- If one thread blocks, the kernel can run another thread from the same or a different process.
- Kernel threads handle blocking system calls and page faults better than user-level threads.
- Some systems recycle kernel threads to reduce creation and destruction overhead.
- The main disadvantage of kernel threads is higher overhead due to system calls.
- Kernel threads do not completely solve issues such as **fork** semantics and signal delivery.
- Hybrid implementations multiplex user-level threads onto kernel-level threads.
- In a hybrid model, the kernel schedules kernel threads, while user-level threads are scheduled on top of them.